



OpenRLHF: Usability-Oriented RL Training from the RLHFer's Perspective

The OpenRLHF Team

Init: 2024.9 / Update: 2025.8

RLHF infra is fundamentally built around the co-optimization of RL algorithms and distributed systems.

Flow of RLHF

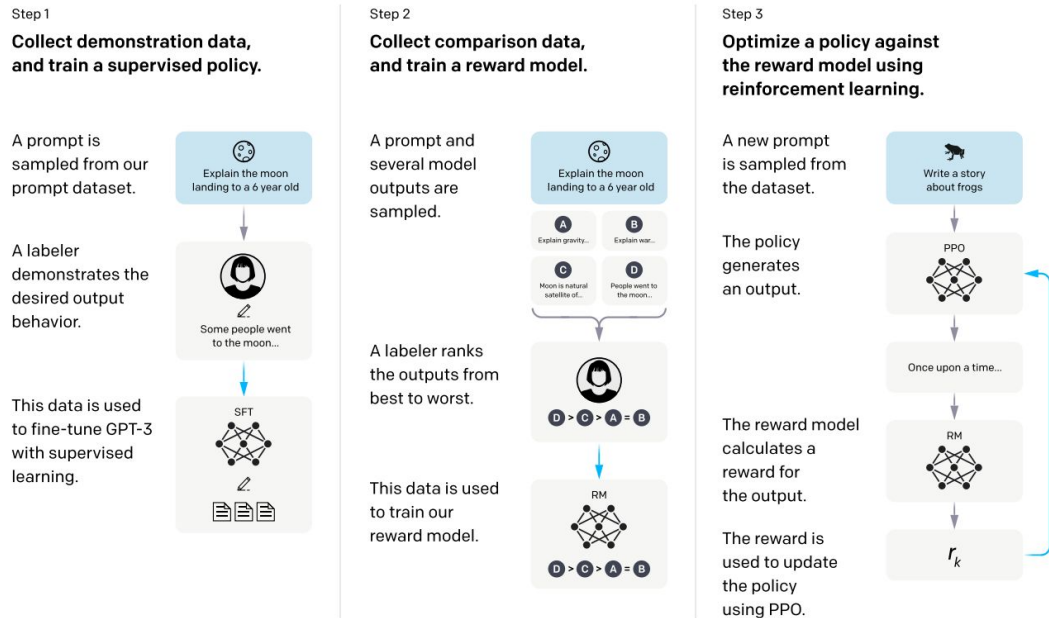
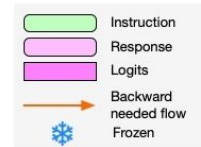
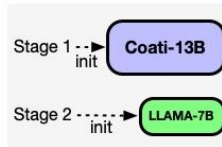


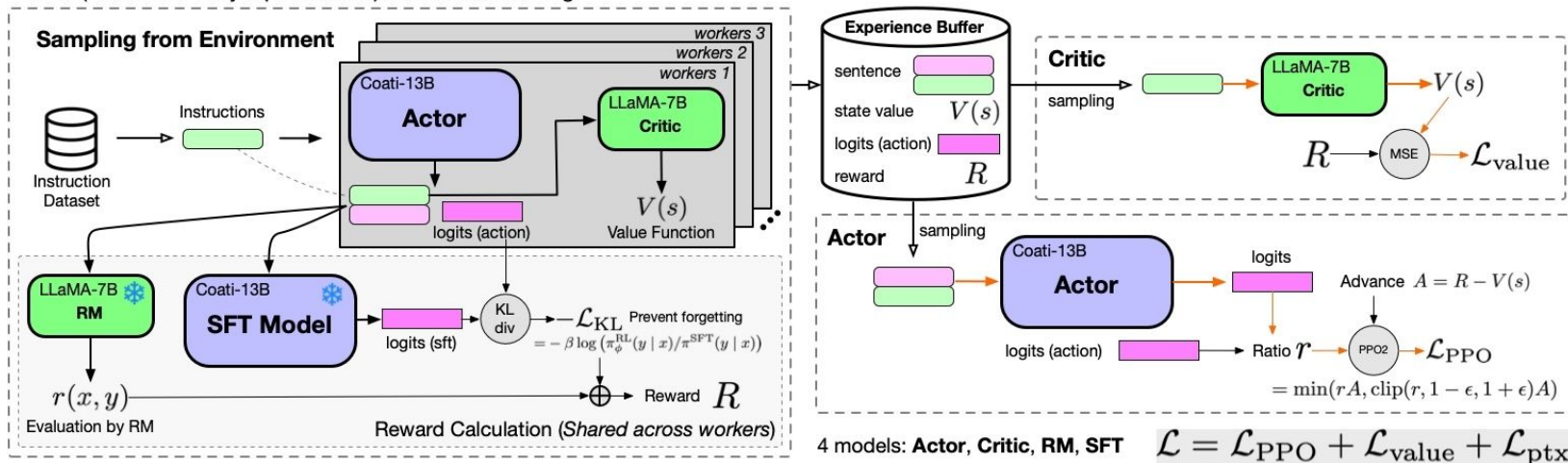
Figure 2: A diagram illustrating the three steps of our method: (1) supervised fine-tuning (SFT), (2) reward model (RM) training, and (3) reinforcement learning via proximal policy optimization (PPO) on this reward model. Blue arrows indicate that this data is used to train one of our models. In Step 2, boxes A-D are samples from our models that get ranked by labelers. See Section 3 for more details on our method.

Flow of RLHF

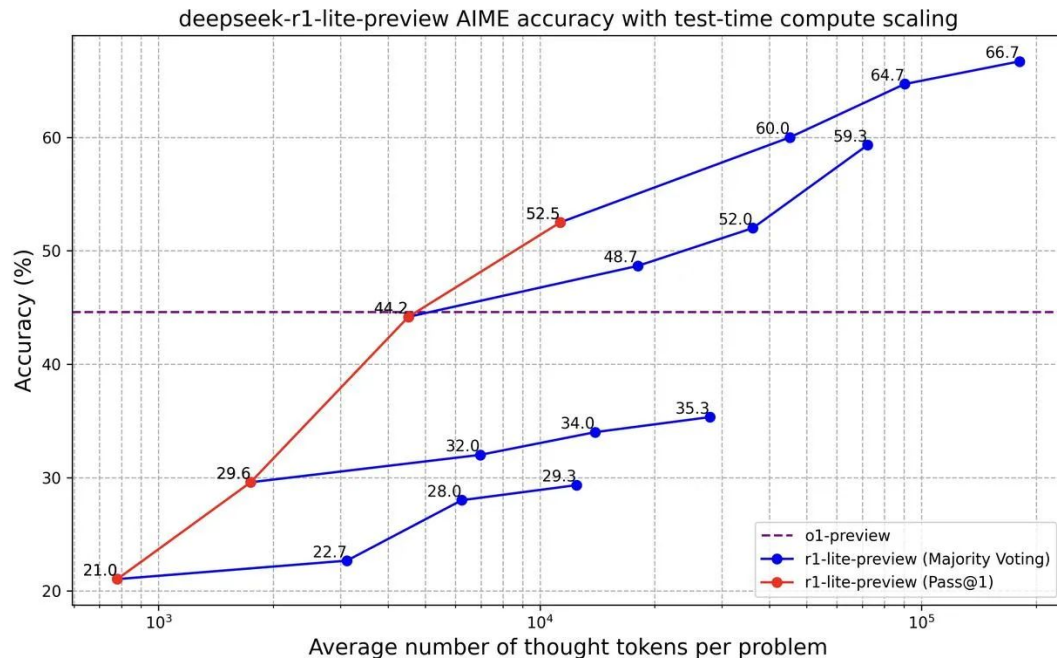
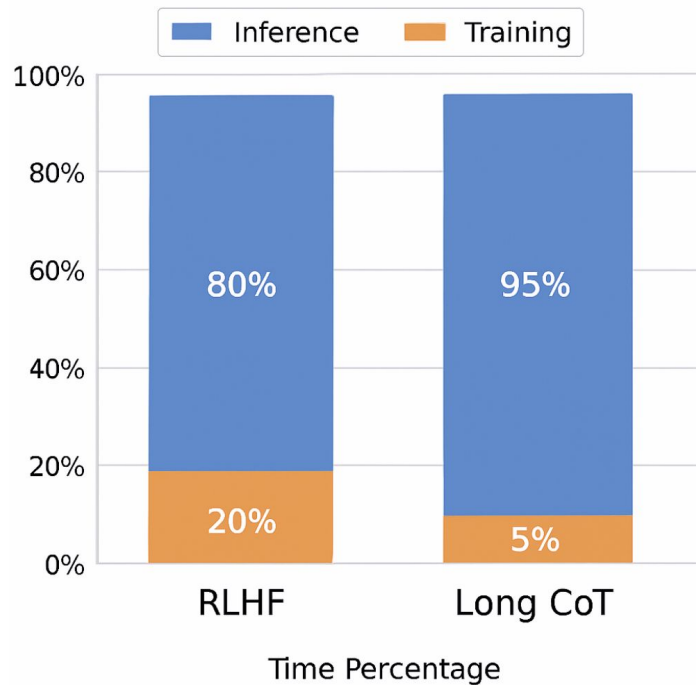
PTX (Pretraining Gradient Mixing): Prevent forgetting



PPO (Proximal Policy Optimization): Reinforce learning



Time breakdown



How to use multi-GPU distributed inference / training in RLHF?

ZeRO with DeepSpeed

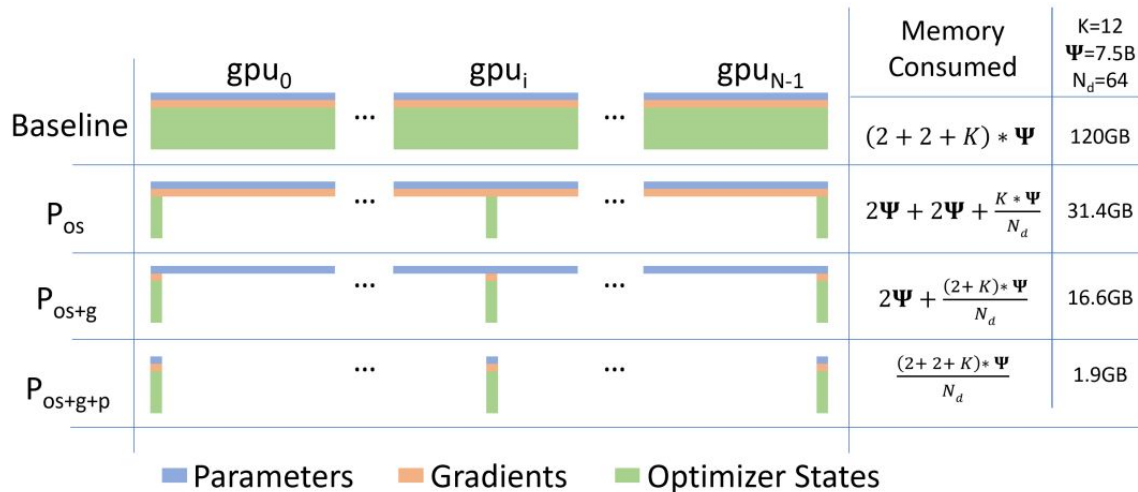
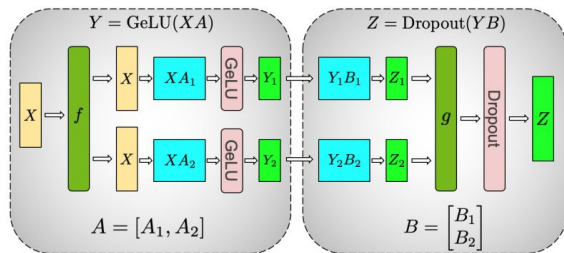


Figure 1: Comparing the per-device memory consumption of model states, with three stages of *ZeRO*-DP optimizations. Ψ denotes model size (number of parameters), K denotes the memory multiplier of optimizer states, and N_d denotes DP degree. In the example, we assume a model size of $\Psi = 7.5B$ and DP of $N_d = 64$ with $K = 12$ based on mixed-precision training with Adam optimizer.

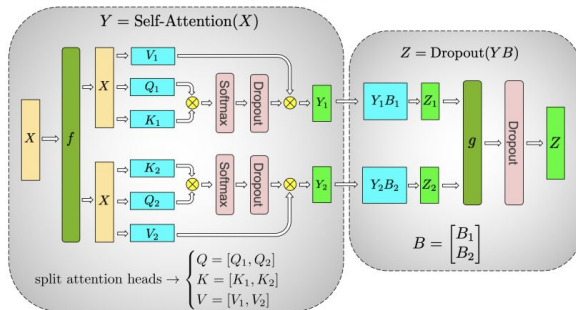
ZeRO with DeepSpeed

LLM generation with ZeRO Stage 3 or FSDP is extremely slow, while ZeRO3 offers acceptable training performance.

3D Parallelism with Megatron



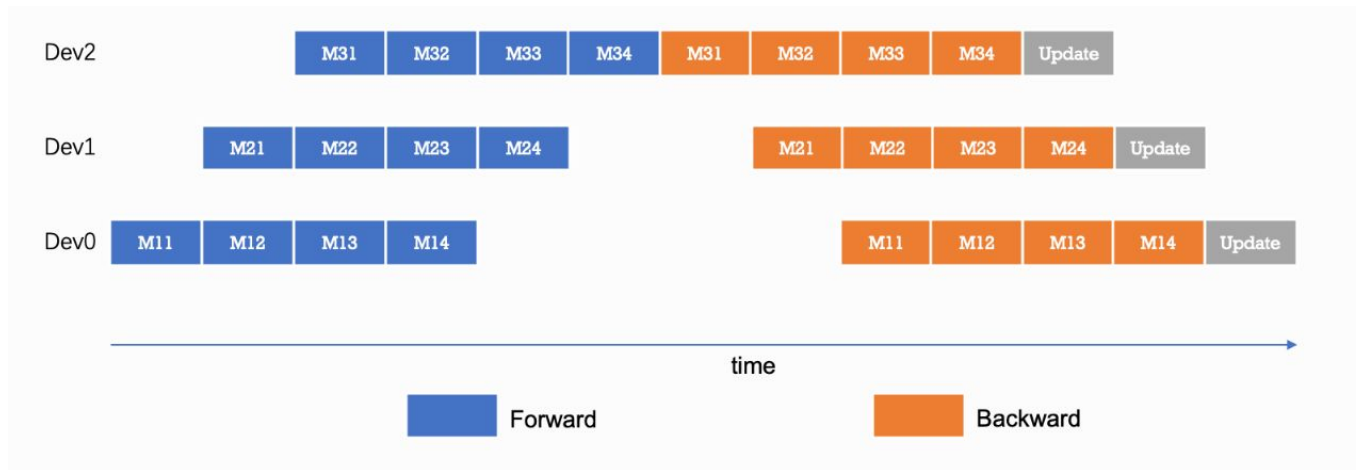
(a) MLP



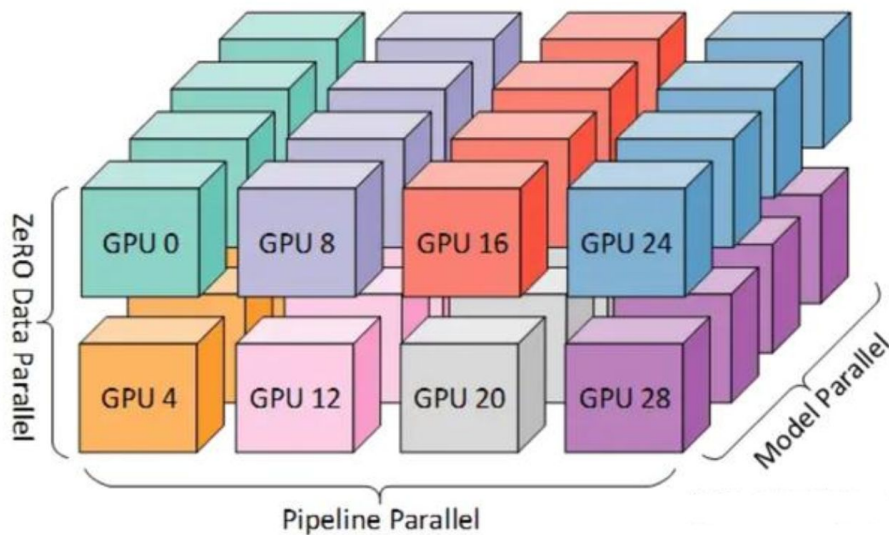
(b) Self-Attention

Figure 3. Blocks of Transformer with Model Parallelism. f and g are conjugate. f is an identity operator in the forward pass and all reduce in the backward pass while g is an all reduce in the forward pass and identity in the backward pass.

3D Parallelism with Megatron-LM



3D Parallelism with Megatron-LM



3D Parallelism with Megatron-LM

LLM generation with Megatron-LM is also quite slow, as Megatron-LM is primarily optimized for training rather than inference.

Megatron-LM is too complex

The Megatron-LM codebase is complex, even with the reinforcement learning loss function defined within its model architecture file. It is difficult for users to make modifications and understanding.

Megatron-LM is too complex

```
class MegatronGPTActorModel(NLPAdapterModelMixin, MegatronGPTModel, AlignableGenerativeInterface):
    def __init__(self, cfg: DictConfig, trainer: Trainer):

        # training calls
    def get_actor_forward_output_and_loss_func(self):
        def fwd_output_and_loss_func(data_iterator, model):
            batch = next(data_iterator)
            response_tokens = batch["response_tokens"]
            advantages = batch["advantages"]
            mask = batch["mask"]
            prev_logprobs = batch["prev_logprobs"]

            attention_mask, _, position_ids = get_ltor_masks_and_position_ids(
                data=response_tokens,
                eod_token=self.tokenizer.eos_id,
                reset_position_ids=False,
                reset_attention_mask=False,
                eod_mask_loss=False,
            )

            batch = {
                "tokens": response_tokens,
                "attention_mask": attention_mask,
                "position_ids": position_ids,
                "advantages": advantages,
                "prev_log_probs": prev_logprobs,
                "mask": mask,
            }

            required_keys = set()
            if parallel_state.get_pipeline_model_parallel_world_size() == 1:
                required_keys.update(batch.keys())
            else:
                required_keys.add("attention_mask")

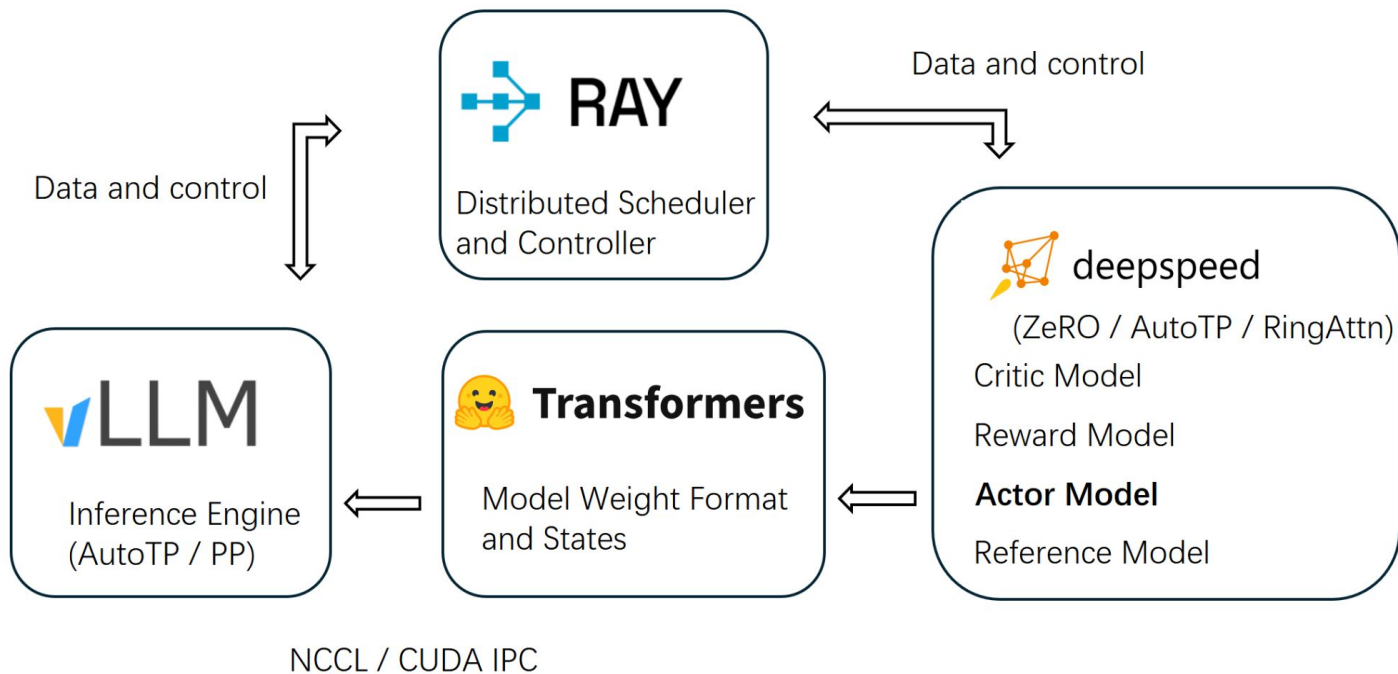
            if parallel_state.is_pipeline_first_stage():
                required_keys.update(("tokens", "position_ids"))

            if parallel_state.is_pipeline_last_stage():
                required_keys.update(("tokens", "advantages", "mask", "prev_log_probs"))
```

https://github.com/NVIDIA/NeMo-Aligner/blob/4749e44dc9806485ca29549d97b8dad7e445759a/nemo_aligner/models/nlp/gpt/megatron_gpt_ppo_actor.py#L58

Ray Architecture of OpenRLHF

The tradeoff between performance and easy-of-use



An introduction to Ray

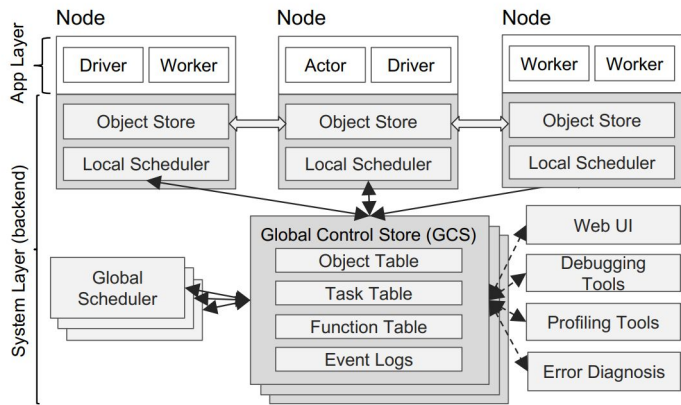


Figure 5: Ray's architecture consists of two parts: an *application* layer and a *system* layer. The application layer implements the API and the computation model described in Section 3, the system layer implements task scheduling and data management to satisfy the performance and fault-tolerance requirements.

```
import ray
from ray.util.placement_group import placement_group

ray.init(address='auto')

# Create Placement Groups
pg = placement_group([{"v100": 1, "h100": 1}])
ray.get([pg.ready()])

@ray.remote
def reference_or_reward_model():
    # Task suitable for small GPU memory
    pass

@ray.remote
def actor_or_critic_model():
    # Task suitable for large GPU memory
    pass

# Launch Tasks
result1 = reference_or_reward_model.options(placement_group=pg, resources={"v100": 1}).remote()
result2 = actor_or_critic_model.options(placement_group=pg, resources={"h100": 1}).remote()
```

Ray Architecture of OpenRLHF

```
@ray.remote(num_gpus=1)
class PolicyModelActor(BaseModelActor):
    def init_model_from_pretrained(self, strategy: DeepspeedStrategy, pretrain, max_steps=None, vllm_engines=None):
        args = strategy.args
        self.save_hf_ckpt = args.save_hf_ckpt
        self.disable_ds_ckpt = args.disable_ds_ckpt
        self.vllm_engines = vllm_engines
        self.max_steps = max_steps

    if getattr(args, "vllm_num_engines", 0) > 0:
        # To prevent hanging during NCCL synchronization of weights between DeepSpeed and vLLM.
        # see https://github.com/vllm-project/vllm/blob/c6b0a7d3ba03ca414be1174e9bd86a97191b7090/vllm/worker/worker.py#L100
        if getattr(args, "vllm_sync_backend", "nccl") == "nccl":
            os.environ["NCCL_CUMEM_ENABLE"] = "0"

    self.setup_distributed(strategy)

    actor = Actor(
        pretrain,
        attn_implementation=strategy.args.attn_implementation,
        bf16=strategy.args.bf16,
        load_in_4bit=strategy.args.load_in_4bit,
        lora_rank=strategy.args.lora_rank,
        lora_alpha=strategy.args.lora_alpha,
        target_modules=strategy.args.target_modules,
        lora_dropout=strategy.args.lora_dropout,
        ds_config=strategy.get_ds_train_config(is_actor=True),
        packing_samples=strategy.args.packing_samples,
        temperature=strategy.args.temperature,
        use liger_kernel=strategy.args.use_liger_kernel,
    )
    strategy.print(actor)
```

```
def ppo_train(self, global_steps):
    status = {}

    # trigger remote critic model training
    if self.critic_model_group is not None:
        # sync for deepspeed_enable_sleep
        if self.strategy.args.deepspeed_enable_sleep:
            ray.get(self.critic_model_group.async_run_method(method_name="reload_states"))

        critic_status_ref = self.critic_model_group.async_run_method(method_name="fit")

    if self.strategy.args.colocate_all_models or self.strategy.args.deepspeed_enable_sleep:
        status.update(ray.get(critic_status_ref)[0])
    if self.strategy.args.deepspeed_enable_sleep:
        ray.get(self.critic_model_group.async_run_method(method_name="offload_states"))

    # actor model training
    if global_steps > self.freezing_actor_steps:
        if self.strategy.args.deepspeed_enable_sleep:
            ray.get(self.actor_model_group.async_run_method(method_name="reload_states"))

        actor_status_ref = self.actor_model_group.async_run_method(method_name="fit", k1_ctl=self.k1_ctl.value)
        status.update(ray.get(actor_status_ref)[0])

    if self.strategy.args.deepspeed_enable_sleep:
        ray.get(self.actor_model_group.async_run_method(method_name="offload_states"))

    # 4. broadcast weights to vllm engines
    if self.vllm_engines is not None:
        self.broadcast_to_vllm()

    # 5. wait remote critic model training done
    if self.critic_model_group and not self.strategy.args.colocate_all_models:
        status.update(ray.get(critic_status_ref)[0])

    return status
```

An introduction to vLLM



vLLM is fast with:

- State-of-the-art serving throughput
- Efficient management of attention key and value memory with **PagedAttention**
- Continuous batching of incoming requests
- Fast model execution with CUDA/HIP graph
- Quantization: [GPTQ](#), [AWQ](#), INT4, INT8, and FP8
- Optimized CUDA kernels, including integration with FlashAttention and FlashInfer.
- Speculative decoding
- Chunked prefill

Weight synchronization with AutoTP

```
def load_weights(self, weights: Iterable[Tuple[str, torch.Tensor]]):
    stacked_params_mapping = [
        # (param_name, shard_name, shard_id)
        (".kv_proj", ".q_proj", "q"),
        (".kv_proj", ".k_proj", "k"),
        (".kv_proj", ".v_proj", "v"),
        (".gate_up_proj", ".gate_proj", 0),
        (".gate_up_proj", ".up_proj", 1),
    ]
    params_dict = dict(self.named_parameters())
    for name, loaded_weight in weights:
        if "rotary_emb.inv_freq" in name:
            continue
        if ("rotary_emb.cos_cached" in name
            or "rotary_emb.sin_cached" in name):
            # Models trained using ColossalAI may include these tensors in
            # the checkpoint. Skip them.
            continue
        # With tie_word_embeddings, we can skip lm_head.weight
        # The weight might appear unnecessarily in the files if the model is
        # processed with quantization, LoRA, fine-tuning, etc.
        if self.config.tie_word_embeddings and "lm_head.weight" in name:
            continue
        if scale_name := get_compressed_tensors_cache_scale(name):
            # Loading kv cache scales for compressed-tensors quantization
            param = params_dict[scale_name]
            weight_loader = getattr(param, "weight_loader",
                                    default_weight_loader)
            loaded_weight = loaded_weight[0]
            weight_loader(param, loaded_weight)
        continue
```

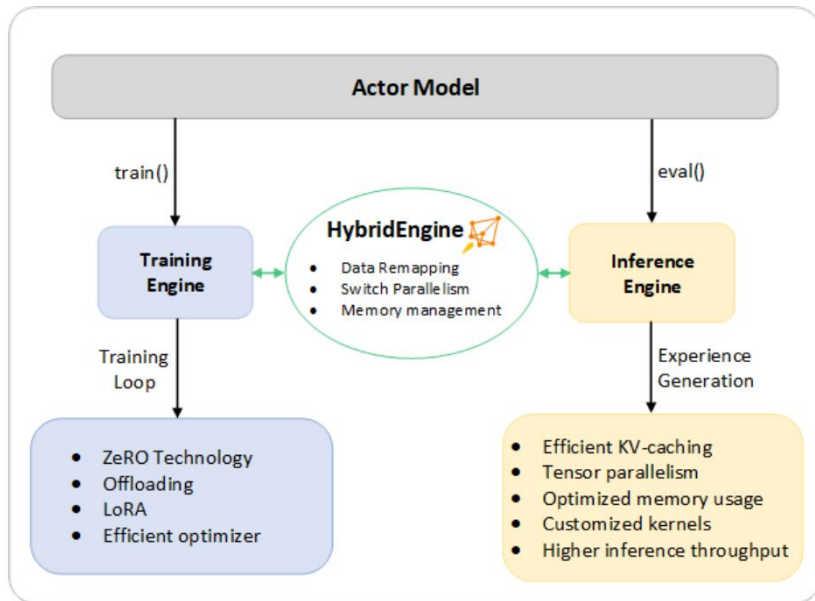
vLLM+AutoTP and DeepSpeed ZeRO/AutoTP can be seamlessly integrated with Hugging Face models.

https://github.com/vllm-project/vllm/blob/9606c7197df073e373ab9e716a62dd4c35398865/vllm/model_executor/models/llama.py#L464

Hugging Face model format

Without the Hugging Face model format to bridge vLLM (AutoTP) and DeepSpeed (ZeRO + AutoTP), synchronizing weights from the Training Engine to the Inference Engine becomes highly complex.

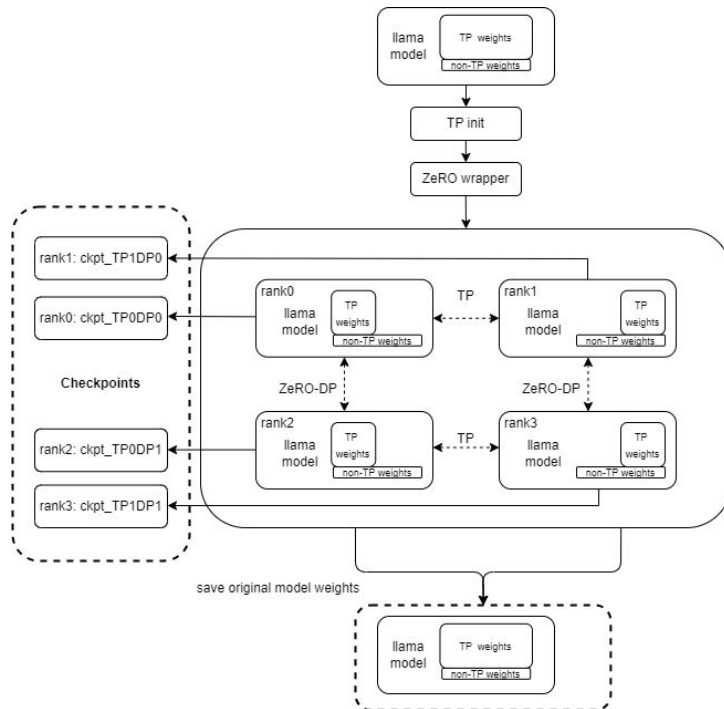
Hybrid Engine



OpenRLHF supports a hybrid engine, enabling all models and vLLM engines to share GPUs and minimize GPU idling.

More details are in: <https://blog.vllm.ai/2025/04/23/openrlhf-vllm.html>

Easy 3D Parallelism with DeepSpeed 0.17.x



<https://github.com/deepspeedai/DeepSpeed/blob/master/blogs/huggingface-tp/README.md>

Ray as training launcher

```
# launch the master node of ray in container
ray start --head --node-ip-address 0.0.0.0 --num-gpus 8

# if you want to launch ray on more nodes, use
ray start --address {MASTER-NODE-ADDRESS}:6379 --num-gpus 8
```

```
ray job submit --address="http://127.0.0.1:8265" \
--runtime-env-json='{ "working_dir": "/openrlhf" }' \
-- python3 -m openrlhf.cli.train_ppo_ray \
--ref_num_nodes 1 \
--ref_num_gpus_per_node 8 \
--reward_num_nodes 1 \
--reward_num_gpus_per_node 8 \
--actor_num_nodes 1 \
--actor_num_gpus_per_node 8 \
--vllm_num_engines 8 \
--vllm_tensor_parallel_size 1 \
--colocate_all_models \
--vllm_gpu_memory_utilization 0.6 \
--advantage_estimator reinforce_baseline \
--pretrain OpenRLHF/Llama-3-8b-sft-mixture \
--reward_pretrain OpenRLHF/Llama-3-8b-rm-700k \
--save_path /openrlhf/examples/test_scripts/final/llama3-8b-rlhf \
--ckpt_path /openrlhf/examples/test_scripts/ckpt/llama3-8b-rlhf \
--save_hf_ckpt \
--micro_train_batch_size 4 \
--train_batch_size 128 \
--micro_rollout_batch_size 8 \
--rollout_batch_size 1024 \
--n_samples_per_prompt 4 \
--max_epochs 1 \
--prompt_max_len 1024 \
--max_samples 20000 \
--generate_max_len 1024 \
--zero_stage 3 \
--bf16 \
--actor_learning_rate 5e-7 \
--critic_learning_rate 9e-6 \
--init_kl_coef 1e-4 \
--prompt_data OpenRLHF/prompt-collection-v0.1 \
--input_key context_messages \
--apply_chat_template \
--normalize_reward \
--gradient_checkpointing \
--packing_samples \
--vllm_sync_backend nccl \
--enforce_eager \
--vllm_enable_sleep \
--deepspeed_enable_sleep
```

Agentic RL

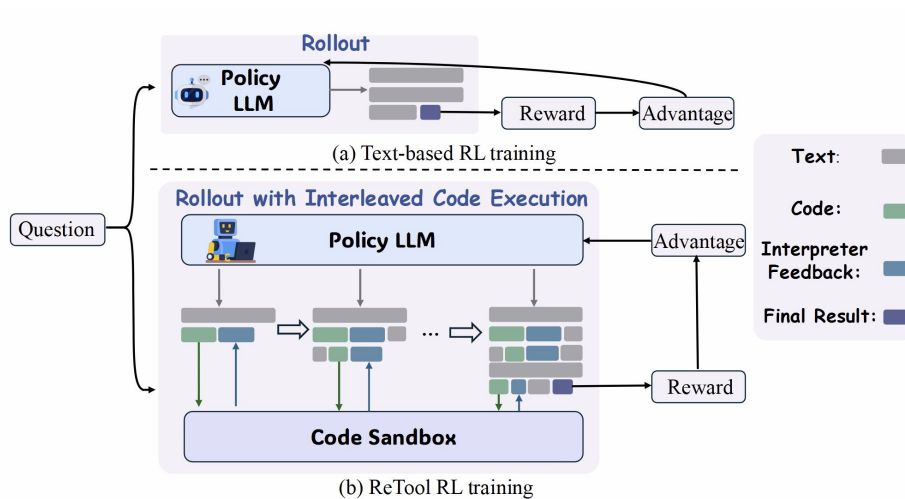
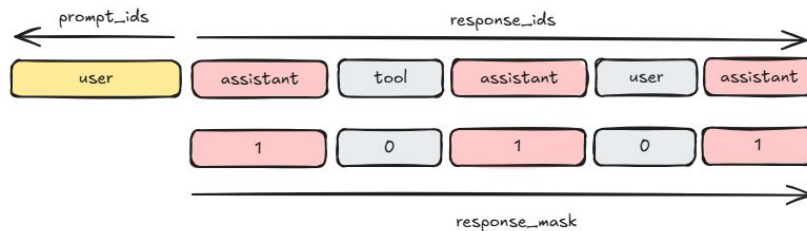
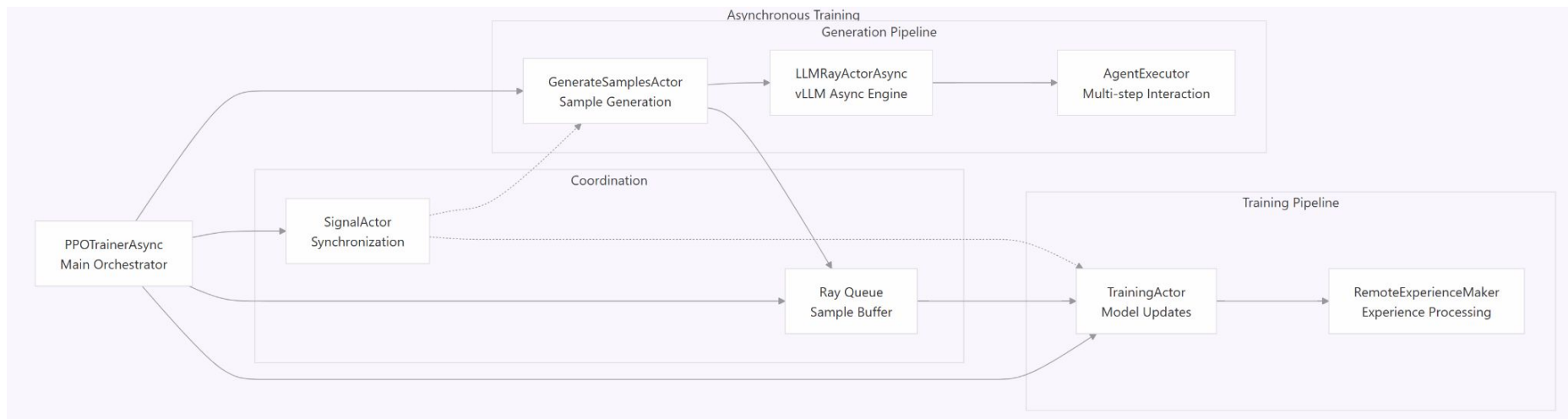


Figure 2 Demonstration of text-based RL training process and ReTool's RL training process.



Async Agentic RL

https://openrlhf.readthedocs.io/en/latest/async_rl.html



Agentic RL & Async RL (Custom Environment)

```
# A simple n-step random environment
class AgentInstance(AgentInstanceBase):
    async def __init__(self, *args, **kwargs):
        self.step_idx = 0
        self.max_steps = random.randint(1, 3) # 1-3 steps

    async def reset(self, states: dict, **kwargs):
        return {"observation": states["observation"]} # Return original text observation

    async def step(self, states: dict, **kwargs) -> Dict[str, Any]:
        print(f"step_idx: {self.step_idx}, max_steps: {self.max_steps}")

        observation_text = states["observation_text"]
        action_text = states["action_text"]
        label = states["label"]

        # Check if episode is done
        done = self.step_idx >= self.max_steps
        reward = torch.randint(0, 2, (1,)).float() if done else torch.tensor(0)

        # Generate environment feedback based on whether episode is done
        environment_feedback = (
            "\n\nHuman: [CORRECT]\n</s>"
            if done
            else "\n\nHuman: [INCORRECT]\nPlease analyze the issues and try again.\n</s>\n\nAssistant:"
        )

        self.step_idx += 1

    return {
        "rewards": reward, # Rewards for advantage calculation
        "scores": reward, # Scores for dynamic filtering (0-1 reward)
        "environment_feedback": environment_feedback, # Environment feedback text
        "done": done, # Boolean indicating if the episode is complete
        "sampling_params": states.get("sampling_params", None), # Parameters for vLLM sampling in
        "extra_logs": {"dummy_scores": reward}, # Additional logging information
    }
```

```
class AgentExecutor(AgentExecutorBase):
    def __init__(self, max_steps, max_length, llm_engine, hf_tokenizer, result_queue):
        super().__init__(AgentInstance, max_steps, max_length, llm_engine, hf_tokenizer, result_queue)

    async def execute(self, prompt, label, sampling_params):
        # You could override the execute function of AgentExecutorBase to add custom agent running logic
        return await super().execute(prompt, label, sampling_params)
```

PPO implementation optimization

Table of contents

0 - Introduction

1 - Advanced Tricks for Training LLM with PPO

1.1 - LLM-specific Tricks

1.2 - PPO-specific Tricks

1.3 - Innovative Strategies

Citation

Reference

<https://hijkzzz.notion.site/rLhf-implementation-tricks>

Implementation of SOTA RL Tricks

- [REINFORCE++-baseline \(-local mean / global std\)](#)
- [Dynamic Sampling](#) (Filter out zero-gradient samples)
- Clip-Higher (Prevent entropy collapse)
- K2 KL Loss ([Unbiased KL estimator](#))
- Length penalty (Prevent overlong output)
- [Truncated Important Sampling](#) (Correct off-policy issues caused by vLLM and training kernels)
- [Token-in-token out generation in Agentic RL](#) (Avoids inconsistency of token IDs during inference and training)
- GSPO/GFPO

<https://hijkzzz.notion.site/prorl-v2>

[Part I: Tricks or Traps? A Deep Dive into RL for LLM Reasoning](#)

REINFORCE++-baseline

1. Proximal Policy Optimization (PPO-Clip) with REINFORCE++-baseline

At ProRL's core is the **clipped PPO loss**, which stabilizes policy updates by restricting how much the new policy can diverge from the old ones:

$$\mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E}_{\tau} \left[\min \left(r_{\theta}(\tau) A(\tau), \text{clip}(r_{\theta}(\tau), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) A(\tau) \right) \right]$$

where:

- $r_{\theta}(\tau) = \frac{\pi_{\theta}(\tau)}{\pi_{\text{old}}(\tau)}$
- $\tilde{R}_{\tau} = R_{\tau} - \mu_{\text{group}}, \mu_{\text{group}} = \text{mean}_{\text{group}}(R_{\tau})$
 $A(\tau) = \frac{\tilde{R}_{\tau} - \mu_{\text{batch}}}{\sigma_{\text{batch}}}, \mu_{\text{batch}} = \text{mean}_{\text{batch}}(\tilde{R}_{\tau}), \sigma_{\text{batch}} = \text{std}_{\text{batch}}(\tilde{R}_{\tau})$

"group" refers to all generated responses for the same prompt. "batch" refers to the rollout global batch.

Global Advantage Normalization (token-level) in the REINFORCE++-baseline helps prevent value instability caused by small group sizes in GRPO. And REINFORCE++-baseline uses the [unbiased K2 estimator](#) to calculate the KL loss.

REINFORCE++-baseline

PPO-GAE(λ) with advantage normalization.

$$\begin{aligned}\delta_t &= r_t + \gamma V(s_{t+1}) - V(s_t), \\ \hat{A}_t &= \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \\ \tilde{A}_t &= \frac{\hat{A}_t - \text{mean}(\hat{A})}{\text{std}(\hat{A}) + \epsilon}\end{aligned}$$

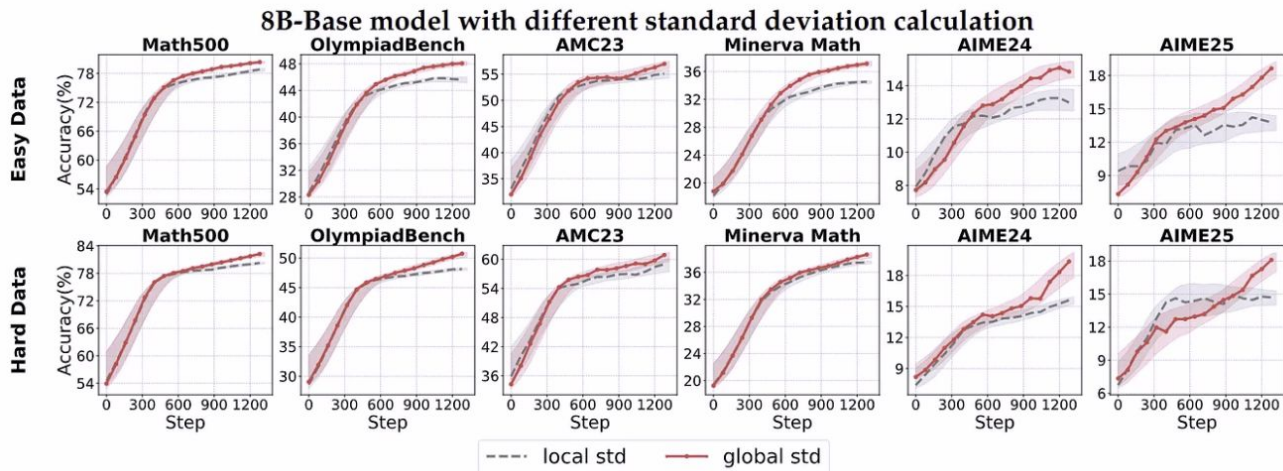
REINFORCE++-baseline sets λ to 1 and substitute $V(s_t)$ with the group mean reward.

$$\begin{aligned}A &= G_\tau - \mu_{\text{group}}, \quad \mu_{\text{group}} = \text{mean}_{\text{group}}(G_\tau) \\ A_{\text{norm}}(\tau) &= \frac{A(\tau) - \mu_{\text{batch}}}{\sigma_{\text{batch}}}, \quad \mu_{\text{batch}} = \text{mean}_{\text{batch}}(A(\tau)), \quad \sigma_{\text{batch}} = \text{std}_{\text{batch}}(A(\tau))\end{aligned}$$

REINFORCE++-baseline only replaces the critic network in PPO with the group reward mean.

REINFORCE++-baseline

	300-step/avg@32/iternum-2(inference)	aime24	aime25	hmmt_feb_2025	hmmt_feb_2024	cimmc	avg
reinforce++	with-baseline	30.83333	27.1875	17.91667	18.95833	25.625	24.10416667
reinforce++	with-baseline,vllm-correction	31.5625	23.9583	20.625	20.1042	29.375	25.125
grpo		31.66667	21.875	16.97917	17.70833	24.6875	22.58333333
ppo		30.20833	21.66667	15	18.4375	23.95833	21.85416667



Industrial Impact

OpenRLHF is the **first** easy-to-use, high-performance open-source RLHF framework **built on Ray, vLLM, ZeRO-3 and HuggingFace Transformers**. It had a profound impact on subsequent open-source RLHF frameworks, **such as Bytedance veRL, Alibaba ROLL, NeMo RL and Zhipu SLIME, even HuggingFace TRL (the vLLM feature)**.

This architecture and paradigm of OpenRLHF have become an industry standard.